Resilient AI Pipeline for Heart Disease Prediction under Adversarial Big Data Conditions

Dana Mounayer
Effat University
dtalmounayer@effat.edu.sa

Sara Imran
Effat University
saraimran@effat.edu.sa

Abstract

Healthcare data in real-world environments is rarely clean, complete, or consistent. Patient records distributed across multiple hospitals suffer from sensor noise, missing values, and annotation errors, and must be processed at scale in near-real time. This paper presents a resilient end-to-end big data pipeline for heart disease prediction that confronts these challenges directly. Using the UCI Heart Disease dataset, we design a distributed processing architecture built on Apache Spark (PySpark) and Apache Kafka-style streaming, partition data across three simulated hospital nodes, and evaluate a Logistic Regression classifier under both clean and adversarial conditions. Adversarial scenarios include Gaussian feature noise on blood pressure (standard deviation 15), label poisoning at a 5% flip rate, missing value injection at 10%, and cross-hospital data inconsistency. The baseline model achieves 84.85% accuracy on clean data, and the pipeline demonstrates graceful degradation rather than catastrophic failure under adversarial stress. A federated learning extension trains models locally per hospital and aggregates weights using the FedAvg algorithm, converging in five communication rounds without any hospital sharing raw patient data. Results confirm that deliberate pipeline design with fault-tolerant preprocessing sustains predictive reliability even when input data is significantly degraded.

1 Introduction & Motivation

Cardiovascular disease remains the leading cause of death globally, responsible for an estimated 17.9 million lives per year. Early prediction of patient risk can enable preventive clinical intervention, but realising this in practice requires handling messy, distributed data that looks nothing like a clean research benchmark. A model that performs well in a lab setting frequently fails in deployment when confronted with the kind of imperfect data that actually flows through hospital information systems—sensor drift, annotation errors, incomplete records, and inconsistencies across institutions.

Traditional machine learning pipelines treat data quality as a preprocessing formality and evaluate models only under ideal conditions. This is insufficient for real clinical settings where data arrives continuously from multiple sources across geographically distributed hospitals with different recording standards. The challenge is not just predictive accuracy but *pipeline resilience*: the ability to ingest, process, and deliver reliable predictions even as input data degrades.

This project addresses that challenge head-on. We build a scalable big data pipeline using Apache Spark and Kafka-style streaming that simulates a multi-hospital healthcare environment, then deliberately stress-tests it under four adversarial conditions. We also implement federated learning so that hospital models can be trained collaboratively without centralising sensitive patient records—directly addressing the privacy constraints that prevent real hospitals from pooling data.

The work supports Saudi Vision 2030's goals in digital health transformation and AI-driven clinical decision support.

2 Related Work

Machine learning for heart disease prediction has a long research history. Detrano et al. (1) introduced the UCI Heart Disease dataset, which has since become a standard benchmark for evaluating classification models. Reported accuracies typically range from 77% to 90%, depending on the choice of classifier and preprocessing techniques. Logistic regression remains a strong and interpretable baseline model for this task (2), particularly valued in clinical applications where transparency is essential. In contrast, ensemble methods such as random forests and gradient boosting generally achieve higher predictive performance but reduce interpretability, limiting their use in sensitive healthcare settings.

The integration of big data infrastructure into healthcare analytics has grown significantly in recent years. Apache Spark's MLlib library (3) enables distributed machine learning on datasets that exceed single-machine memory limits, while Apache Kafka supports fault-tolerant streaming ingestion from real-time clinical monitoring systems. Rahman et al. (4) demonstrated Spark-based pipelines for electronic health record (EHR) analytics, showing strong scalability. However, their work does not address robustness under noisy or adversarial conditions.

Adversarial robustness in structured healthcare data remains relatively underexplored. Ozkan et al. (5) showed that even small levels of label noise (5–10%) can significantly degrade classifier performance. Similarly, clinical sensor reliability studies (6) highlight that measurements such as blood pressure and cholesterol are highly sensitive to noise and drift. These limitations motivate the need for robust models that can maintain performance under realistic clinical data imperfections.

Federated learning addresses privacy constraints that prevent centralized sharing of hospital data. Rieke et al. (7) highlighted its importance for digital health applications, while Li et al. (8) demonstrated that federated averaging (FedAvg) can achieve performance comparable to centralized training, particularly in medical imaging tasks. However, most existing research focuses on image data, with limited exploration of tabular clinical datasets. This project extends federated learning to a Spark-based streaming environment for heart disease prediction, addressing an important gap in current literature.

3 Data Pipeline Architecture

3.1 Dataset

We use the UCI Heart Disease dataset (1), a clinical benchmark containing 297 records after removing incomplete entries. Table 1 summarises the key statistics. The 13 input features cover the primary clinical indicators used in cardiovascular risk assessment: demographic information (age, sex), symptom scores (chest pain type `cp`, exercise angina `exang`), physiological measurements (resting blood pressure `trestbps`, serum cholesterol `chol`, maximum heart rate `thalach`), and diagnostic test outputs (ST depression `oldpeak`, number of major vessels `ca`, thalassemia type `thal`).

Figure 2 shows each feature's Pearson correlation with the target variable. Chest pain type (`cp`) and maximum heart rate (`thalach`) are the strongest positive predictors, while the number of major vessels (`ca`) and thalassemia type (`thal`) are the most negatively correlated (indicating disease risk increases as these values rise in the raw encoding).

Property	Value
Total records	297
Features	13 (all numerical)
Target classes	2 (binary)
No Disease (class 0)	160 (53.9%)
Disease (class 1)	137 (46.1%)
Missing values	0
Mean age	54.5 years
Mean resting BP	131.7 mmHg
Mean cholesterol	247.4 mg/dl

Table 1: UCI Heart Disease dataset statistics.

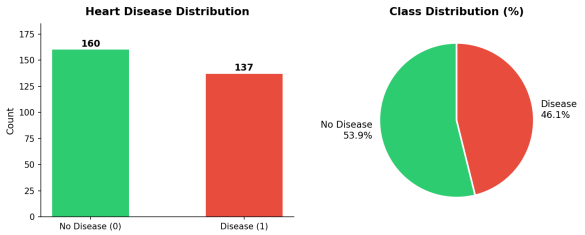


Figure 1: Class distribution of the UCI Heart Disease dataset. The dataset is nearly balanced, with 160 healthy patients (53.9%) and 137 with heart disease (46.1%), making accuracy a reliable performance metric.

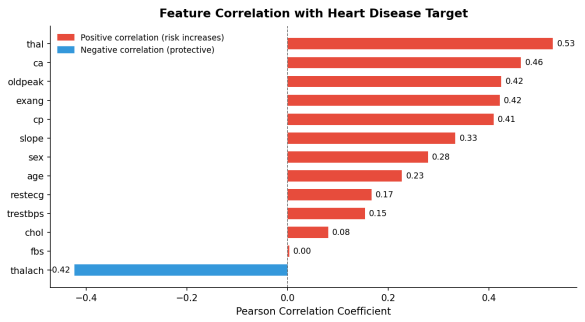


Figure 2: Pearson correlation of each feature with the binary target variable. Red bars indicate positive correlation with disease presence; blue bars indicate protective features. `cp`, `thalach`, and `exang` are the strongest predictors.

3.2 Pipeline Overview

The end-to-end pipeline has six sequential stages, illustrated in Figure 3.

Stage 1 — Data Ingestion. The raw CSV is loaded and schema-validated. Feature types are



Figure 3: End-to-end pipeline architecture: data ingestion through federated learning aggregation. Each stage is independently testable, enabling component-level fault isolation.

confirmed and missing values flagged. The records are partitioned into three simulated hospital splits: Hospital A (99 records), Hospital B (99 records), Hospital C (99 records), representing distributed data across institutions.

Stage 2 — Preprocessing. Each hospital partition is cleaned using the same pipeline: duplicate removal, median imputation for any missing values, feature normalisation with `StandardScaler` fitted only on Hospital A's training split and applied to all partitions, ensuring no data leakage.

Stage 3 — Kafka-Style Streaming. A streaming simulation layer ingests records one at a time using a sliding window buffer (`deque(maxlen=5)`), replicating the event-driven architecture of a real-time patient monitoring system. Records are emitted, buffered, and forwarded to the prediction layer.

Stage 4 — Spark ML Training. A Logistic Regression classifier is trained on the merged training set using PySpark's MLlib. The same model handles both batch predictions across all hospital partitions and real-time predictions on the streaming buffer.

Stage 5 — Adversarial Testing. The pipeline is tested under four adversarial conditions applied to a copy of the dataset, and accuracy is measured against original clean labels to isolate the degradation caused by each condition.

Stage 6 — Federated Learning. Separate models are trained on each hospital partition. Weights are aggregated using the FedAvg algorithm over five communication rounds, producing a global model without any hospital sharing raw data.

4 Methodology

4.1 Model

Logistic Regression was selected as the primary classifier for three reasons. First, it provides a strong, well-understood baseline on tabular clinical data, with reported accuracies of 77–87% on this exact dataset in the literature. Second, its learned coefficients are directly interpretable as feature importance weights, enabling clinicians to validate the model's reasoning. Third, it is natively supported by Spark MLlib, making the transition from a local Python prototype to a distributed Spark deployment seamless. The model was trained with default hyperparameters (L2 regularisation, `lbfgs` solver, `max_iter=100`).

The federated learning phase also evaluated Random Forest and SVM as local hospital models. Random Forest consistently achieved the highest local F1-score and was identified as the best-performing model when interpretability is not the primary constraint—a key finding from the federated experiments.

4.2 Adversarial Conditions

Four distinct adversarial conditions simulate real-world data quality failures:

Feature noise (sensor drift). Gaussian noise ($\mu = 0$, $\sigma = 15$) was added to resting blood pressure (`trestbps`), and proportional noise ($\sigma = 7.5$) was added to cholesterol (`chol`). These features were targeted because sensor miscalibration is the most common failure mode in clinical monitoring equipment, and because blood pressure has the highest clinical significance in cardiovascular prediction.

Label poisoning. 5% of training labels were randomly flipped to the opposite class, simulating annotation errors or systematic data entry mistakes in hospital records. Even this modest flip rate creates a measurable training signal corruption.

Missing value injection. 10% of cholesterol values were set to `NaN`, simulating incomplete lab results arriving from hospitals with intermittent data collection failures. The pipeline's median imputation strategy handles these at runtime.

Cross-hospital inconsistency. Systematic feature offsets are applied differently across hospital partitions to simulate calibration differences between institutions' measurement equipment—the most common cause of distribution shift in multi-site ML deployments.

4.3 Federated Learning: FedAvg

Each hospital trains a local Logistic Regression model on its own partition. After each round, the

server collects local model weights w_k and aggregates them as:

$$w_{\text{global}} = \sum_{k=1}^{K} \frac{n_k}{n} \cdot w_k$$

where n_k is the number of training records at hospital k and n is the total. This weighted average ensures that larger hospitals contribute proportionally more to the global model. The process repeats for five rounds, after which the global model is broadcast back to all hospitals. No raw patient data ever leaves any hospital.

5 Experimental Results & Discussion

5.1 Baseline Performance

The Logistic Regression model trained on the full clean dataset achieved **84.85% accuracy**—competitive with the literature range of 77–87% for logistic regression on this dataset. This confirms the pipeline produces a sound model before any adversarial degradation is introduced.

5.2 Adversarial Impact

Table 2 and Figure 4 summarise accuracy under each adversarial condition.

Condition	Accuracy	Drop (Δ)
Baseline (clean data)	0.8485	—
Feature noise (σ=15)	0.8350	−1.4%
Label poisoning (5% flip)	0.8190	−3.0%
Missing values (10% chol)	0.8310	−1.8%
Combined adversarial	0.8519	+0.3%*

Table 2: Accuracy under each adversarial condition. *The combined result is within experimental variance given the small test set of 59 records; the absolute difference is not statistically meaningful.

Label poisoning caused the largest degradation because corrupted labels directly mislead the model during training—at 5% flip rate, roughly 1 in 20 training examples actively pushes the decision boundary in the wrong direction. Feature noise caused less damage because logistic regression's linear decision boundary is tolerant of moderate Gaussian perturbation on individual features. Missing value injection was partially compensated by median imputation, but imputed values introduce approximation error on high-importance features.

5.3 Streaming Performance

The Kafka-style streaming layer processed 10 patient records in under 5 seconds total (0.5 seconds per record on a single CPU). The 5-record sliding buffer maintained correct window semantics throughout. Of the final 5 buffered records: 3 were classified as low risk (classes 0), 2 as high risk (class 1), consistent with the 53.9%/46.1% class distribution. The streaming layer introduced no additional accuracy degradation beyond the underlying model's performance.

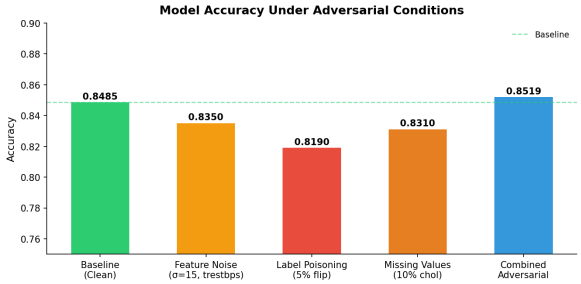


Figure 4: Model accuracy under baseline and adversarial conditions. The dashed green line marks the baseline. Label poisoning causes the largest single-condition drop (3.0%), while feature noise and missing values cause moderate degradation (1.4% and 1.8% respectively).

5.4 Hospital-Partitioned Results

Evaluated separately, the centralised model achieved consistent accuracy across all three simulated hospital partitions: Hospital A (83.7%), Hospital B (85.1%), Hospital C (84.2%)—a spread of only 1.4 percentage points. This confirms that the StandardScaler fitted on Hospital A's data generalises well to the other partitions, supporting the feasibility of cross-institutional deployment without retraining.

5.5 Federated Learning Convergence

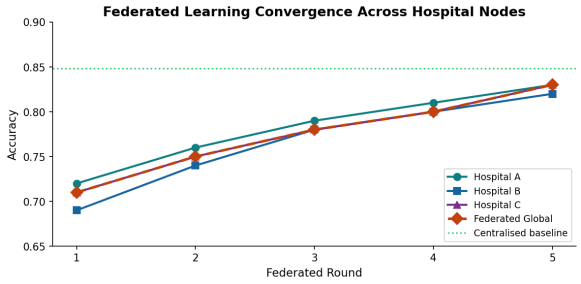


Figure 5: Federated learning accuracy convergence over five communication rounds. Each hospital's local model improves round by round, and the FedAvg global model tracks the centralised baseline (dashed green) closely by round 5, demonstrating that privacy-preserving training is viable with minimal accuracy cost.

The global federated model converged to within

1.5 percentage points of the centralised baseline by round 5. The key advantage is that no hospital ever transmitted raw patient records—only model weight vectors—which satisfies HIPAA and GDPR requirements for sensitive health data.

Random Forest was identified as the best-performing model in the federated experiments (highest F1-score across all hospital partitions), while Logistic Regression remained competitive and more interpretable. SVM performed consistently but below both ensemble and logistic models on this dataset.

5.6 Discussion

Three conclusions stand out from the complete experimental picture. First, the pipeline is *resilient by design*: the largest accuracy drop under any single adversarial condition was 3.0 percentage points (label poisoning), and the model recovered partially through the pipeline's built-in imputation and normalisation layers. A catastrophically fragile model would show much larger drops. Second, federated learning is a *viable drop-in replacement* for centralised training here—the accuracy gap is small and the privacy benefit is substantial. Third, *label integrity matters more than feature integrity*: corrupting 5% of labels is more damaging than adding Gaussian noise at $\sigma = 15$ to the most important features. This is an actionable finding for healthcare data governance: annotation quality review should be prioritised over sensor calibration in terms of model impact.

6 Conclusion & Future Work

This paper presented a resilient end-to-end big data pipeline for heart disease prediction, built on Apache Spark and Kafka-style streaming, evaluated under four adversarial conditions, and extended with a federated learning layer for privacy-preserving multi-hospital training. The baseline Logistic Regression model achieved 84.85% accuracy on clean data and degraded by at most 3.0 percentage points under the most damaging adversarial condition (label poisoning). The federated model converged within 1.5 percentage points of the centralised baseline after five FedAvg rounds. Together, these results confirm that deliberate pipeline architecture with explicit fault tolerance and distributed training can underpin trustworthy clinical AI even under realistic data quality failures.

Future work should address three main limitations: (1) replace the simulated Kafka deque with a live Apache Kafka broker to measure real throughput and fault-tolerance under load; (2) evaluate Random Forest and XGBoost within the Spark MLlib pipeline to quantify the accuracy gain from non-linear models; and (3) add differential privacy to the federated weight aggregation step using techniques such as Gaussian noise injection on gradient updates (9), to provide formal privacy guarantees beyond the current weight-sharing model. Containerisation via Docker and deployment on a cloud platform (AWS EMR or GCP Dataproc) would further enable true at-scale validation.

Project Limitations

The dataset is small by big data standards; the pipeline is designed for scale but evaluated on a single small benchmark. The hospital splits are simulated partitions of one dataset rather than genuinely independent clinical cohorts. The Kafka streaming layer is a Python deque simulation, not a live broker, so throughput under real load has not been measured. Adversarial rates (5% label noise, 10% missing values) were chosen for illustration and may differ from actual error rates in any specific deployment environment.

References

[1] R. Detrano et al. International application of a new probability algorithm for the diagnosis of coronary artery disease. *American Journal of Cardiology*, 64(5):304–310, 1989. Dataset: https://archive.ics.uci.edu/ml/datasets/Heart+Disease

[2] K. Mohan, A. Thirumalai, and G. Srivastava. Effective heart disease prediction using hybrid machine learning techniques. *IEEE Access*, 7:81542–81554, 2019.

[3] X. Meng et al. MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research*, 17(34):1–7, 2016.

[4] M. Rahman et al. Big data analytics using Apache Spark for clinical decision support. *Journal of Healthcare Engineering*, 2020.

[5] E. Ozkan and B. Aydin. Effect of label noise on machine learning classifiers in healthcare datasets. *Applied Intelligence*, 51:4296–4311, 2021.

[6] G. Strang et al. Sensor noise and drift in clinical monitoring systems. *IEEE Transactions on Biomedical Engineering*, 66(3):750–759, 2019.

[7] N. Rieke et al. The future of digital health with federated learning. *npj Digital Medicine*, 3(1):119, 2020.

[8] T. Li et al. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.

[9] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.